Таблицы в pandas

Шпаргалка



Методы из урока «Таблицы в pandas» и ещё немного — по задачам: что нужно сделать, чем, как это выглядит. Под каждой строкой пример; после стрелки — его значение, снятое запуском, если оно короткое. Все примеры — на этих таблицах:

```
import numpy as np
import pandas as pd
days = ["2026-10-01", "2026-10-01",
        "2026-10-02", "2026-10-03"]
df = pd.DataFrame({
    "city": ["Пермь", "Сочи", "Пермь", "Тула"],
    "t": [3.5, 14.0, np.nan, -2.0],
    "day": pd.to_datetime(days),
})
cities = pd.DataFrame({"city": ["Пермь", "Сочи"],
                       "region": ["Урал", "Юг"]})
c2 = cities.rename(columns={"city": "name"})
s = pd.Series([3, 1, 3, 2])
raw = pd.Series(["12", "7.5", "нет"])
df.to_csv("pogoda.csv", index=False)
print(df)
```

```
    city     t        day
0  Пермь   3.5 2026-10-01
1   Сочи  14.0 2026-10-01
2  Пермь   NaN 2026-10-02
3   Тула  -2.0 2026-10-03
```

Прочитать

`pd.read_csv` — таблица из файла CSV

```
pd.read_csv("pogoda.csv").shape → (4, 3)
```

`sep=";", decimal=","` — так пишет русский Excel

```
pd.read_csv("e.csv", sep=";", decimal=",")
```

`encoding=` — файл не в UTF-8: чаще всего `"cp1251"`

```
pd.read_csv("w.csv", encoding="cp1251")
```

`nrows=` — только первые строки; `usecols=` — столбцы

```
pd.read_csv("pogoda.csv", nrows=2).shape → (2, 3)
```

`dtype=` — тип столбцов сразу при чтении

```
pd.read_csv("pogoda.csv", dtype={"t": str})
```

`na_values=` — что ещё считать пропуском

```
pd.read_csv("pogoda.csv", na_values=["нет"])
```

`pd.DataFrame` — таблица из словаря столбцов

```
pd.DataFrame({"x": [1, 2]}).shape → (2, 1)
```

Посмотреть

`df.head` — первые строки, пять по умолчанию; `df.tail()` — последние

```
df.head(2)
```

`df.shape` — строк и столбцов

```
df.shape → (4, 3)
```

`df.dtypes` — типы столбцов

```
df.dtypes["t"] → float64
```

`df.columns` — имена столбцов

```
df.columns.tolist() → ['city', 't', 'day']
```

`df.describe` — сводка: сколько, среднее, разброс, квартили

```
df["t"].describe()["max"] → 14.0
```

`df.isna` — маска пропусков; с `.sum()` — сколько по столбцам

```
df.isna().sum()["t"] → 1
```

`s.value_counts` — сколько раз каждое значение

```
df["city"].value_counts()["Пермь"] → 2
```

`s.nunique` — сколько различных; `s.unique()` — сами они

```
df["city"].nunique() → 3
```

Почистить

`.str.strip` — снять пробелы по краям

```
pd.Series([" Сочи "]).str.strip()[0] → Сочи
```

`.str.lower` — в строчные; `.str.upper()` — в заглавные

```
df["city"].str.lower()[1] → сочи
```

`.str.replace` — замена в строках

```
pd.Series(["7,5"]).str.replace(",", ".")[0] → 7.5
```

`.str.contains` — маска «содержит»

```
df["city"].str.contains("ер").sum() → 2
```

`.str.split` — разбить; `expand=True` — в столбцы

```
pd.Series(["a-b"]).str.split("-")[0] → ['a', 'b']
```

`.str[:k]` — первые k знаков

```
df["city"].str[:2][1] → Со
```

`pd.to_numeric` — в числа; `errors="coerce"` — непонятное в пропуск

```
pd.to_numeric(raw, errors="coerce")[1] → 7.5
```

`s.astype` — сменить тип

```
df["t"].astype(str)[0] → 3.5
```

`s.fillna` — заполнить пропуски

```
df["t"].fillna(0).sum() → 15.5
```

`df.dropna` — убрать строки с пропусками

```
len(df.dropna()) → 3
```

`df.duplicated` — повторы строк; `subset=` — по столбцам

```
df.duplicated(subset=["city"]).sum() → 1
```

`df.drop_duplicates` — убрать повторы

```
len(df.drop_duplicates(subset=["city"])) → 3
```

`df.rename` — переименовать столбцы

```
df.rename(columns={"t": "temp"}).columns[1] → temp
```

`df.drop` — убрать столбцы

```
df.drop(columns=["day"]).shape → (4, 2)
```

`s.replace` — заменить значения по словарю

```
s.replace({3: 0}).tolist() → [0, 1, 0, 2]
```

`s.map` — значение по словарю; чего нет — пропуск

```
s.map({1: "a", 2: "b", 3: "c"})[0] → c
```

Выбрать

`df[["a", "b"]]` — несколько столбцов; `df["a"]` — один, это Series

```
df[["city", "t"]].shape → (4, 2)
```

`df[mask]` — строки по маске

```
len(df[df["t"] > 0]) → 2
```

`&` — «и»; ещё `|` — «или», `~` — «не»; условия в скобках

```
((df["t"] > 0) & (df["t"] < 10)).sum() → 1
```

`s.isin` — маска «равно одному из»

```
df["city"].isin(["Сочи", "Тула"]).sum() → 2
```

`s.between` — между, концы входят

```
df["t"].between(0, 10).sum() → 1
```

`df.loc` — по меткам: `loc[строки, столбцы]`; строки и маской

```
df.loc[0, "city"] → Пермь
```

`df.iloc` — по номерам, как в numpy

```
df.iloc[-1, 0] → Тула
```

`df.query` — маска, записанная строкой

```
len(df.query("t > 0 and city != 'Сочи'")) → 1
```

`df.nlargest` — k строк с наибольшим; `nsmallest` — с наименьшим

```
df.nlargest(1, "t")["city"].iloc[0] → Сочи
```

`df.sample` — случайные строки

```
df.sample(2, random_state=0).shape → (2, 3)
```

Добавить и изменить

`df.assign` — копия с новым столбцом; на месте — `df["f"] = ...`

```
df.assign(f=df["t"] * 1.8 + 32).shape → (4, 4)
```

`np.where` — столбец «если, то, иначе»

```
np.where(df["t"] > 0, 1, 0) → [1 1 0 0]
```

`df.loc[mask, col] =` — записать по маске; цепочка `df[m][c] = ...` не пишет

```
d = df.copy(); d.loc[d["t"] < 0, "t"] = 0
```

`pd.cut` — числа по интервалам

```
pd.cut(df["t"], [-10, 0, 20]).iloc[0] → (0, 20]
```

`s.rank` — ранги; `method=` — как делить равные

```
s.rank().tolist() → [3.5, 1.0, 3.5, 2.0]
```

`s.clip` — обрезать до границ

```
s.clip(2, 3).tolist() → [3, 2, 3, 2]
```

`s.apply` — своя функция на каждый элемент: цикл питона, для чисел медленнее столбца в десятки раз; сначала `np.where`, `.map`, `.str`

```
s.apply(lambda v: v % 2).sum() → 3
```

`df.apply(f, axis=1)` — `f` получает строку, `Series` одного типа: целое рядом с дробным станет дробным; без `axis` — столбец

```
df.apply(lambda r: r["t"] * 2, axis=1).sum() → 31.0
```

Сгруппировать

`.groupby().size` — сколько строк в каждой группе

```
df.groupby("city").size()["Пермь"] → 2
```

`.groupby()[col].mean` — счёт столбца по группам; `sum`, `max`, `count`

```
df.groupby("city")["t"].mean()["Пермь"] → 3.5
```

`.apply` — подтаблица группы без столбца группировки (в pandas 2 он был); ключ — `g.name`; число проще через `agg`

```
df.groupby("city").apply(len)["Пермь"] → 2
```

`.agg` — несколько счётов, каждый под своим именем

```
df.groupby("city").agg(m=("t", "max"))
```

`.transform` — значение группы в каждой строке

```
df.groupby("city")["t"].transform("max")[2] → 3.5
```

`.nunique` — сколько различных в группе

```
df.groupby("city")["day"].nunique()["Пермь"] → 2
```

`.cumcount` — номер строки в группе, с нуля

```
df.groupby("city").cumcount()[2] → 1
```

`.first` — первое значение группы; `.last()` — последнее

```
df.groupby("city")["t"].first()["Сочи"] → 14.0
```

`s.idxmax` — метка строки с наибольшим

```
df["t"].idxmax() → 1
```

`groupby([a, b])` — группы по паре столбцов

```
df.groupby(["city", "day"]).size().shape → (4,)
```

`normalize=True` — доли вместо количеств, у `value_counts`

```
s.value_counts(normalize=True)[3] → 0.5
```

Соединить

`df.merge` — строки двух таблиц рядом по ключу

```
df.merge(cities, on="city").shape → (3, 4)
```

`how=` — `"inner"` — общие ключи, `"left"` — все слева

```
df.merge(cities, how="left").shape → (4, 4)
```

`left_on=, right_on=` — ключи с разными именами

```
df.merge(c2, left_on="city", right_on="name")
```

`indicator=True` — столбец `_merge`: откуда строка

```
df.merge(cities, how="left", indicator=True)
```

`validate=` — проверить, что ключ справа не повторяется

```
df.merge(cities, validate="many_to_one")
```

`pd.concat` — таблицы друг под другом

```
pd.concat([df, df]).shape → (8, 3)
```

Даты

`pd.to_datetime` — строки в даты; `format=` — всегда

```
pd.to_datetime("3.9.26", format="%d.%m.%y").day → 3
```

`.dt.day` — части дат; ещё `.dt.year`, `.dt.month`, `.dt.hour`

```
df["day"].dt.day.tolist() → [1, 1, 2, 3]
```

`.dt.dayofweek` — день недели, 0 — понедельник

```
df["day"].dt.dayofweek[0] → 3
```

`.dt.days` — промежуток в днях

```
(df["day"] - df["day"].min()).dt.days.max() → 2
```

`pd.Timedelta` — промежуток; `pd.Timestamp` — момент

```
pd.Timedelta("2 days") / pd.Timedelta("1h") → 48.0
```

`pd.date_range` — все дни подряд

```
len(pd.date_range("2026-09-01", "2026-09-30")) → 30
```

`.dt.strftime` — дату в строку по формату

```
df["day"].dt.strftime("%d.%m")[0] → 01.10
```

`df.resample` — по периодам: `"D"`, `"W"`, `"MS"`

```
df.resample("D", on="day")["t"].mean().size → 3
```

`>= "2026-10-02"` — сравнить со строкой-датой

```
(df["day"] >= "2026-10-02").sum() → 2
```

Сдвиги и окна

`s.shift` — значение из предыдущей строки

```
s.shift().tolist() → [nan, 3.0, 1.0, 3.0]
```

`s.diff` — разность с предыдущей строкой

```
s.diff().tolist() → [nan, -2.0, 2.0, -1.0]
```

`s.pct_change` — изменение в долях

```
s.pct_change()[1].round(2) → -0.67
```

`.rolling(w).mean` — скользящее среднее по w строкам

```
s.rolling(2).mean().tolist() → [nan, 2.0, 2.0, 2.5]
```

`min_periods=` — считать и по неполному окну

```
s.rolling(2, min_periods=1).sum()[0] → 3.0
```

`s.expanding` — окно от начала до текущей строки

```
s.expanding().mean().iloc[-1] → 2.25
```

`s.cumsum` — накопленная сумма; ещё `cummax`, `cummin`

```
s.cumsum().tolist() → [3, 4, 7, 9]
```

`.groupby().shift` — сдвиг внутри групп

```
df.groupby("city")["t"].shift()[2] → 3.5
```

Сводные и форма

`pd.crosstab` — подсчёт пар значений

```
pd.crosstab(df["city"], df["day"]).shape → (3, 3)
```

`df.pivot_table` — функция по парам: `values=`, `aggfunc=`, `fill_value=`

```
df.pivot_table(index="city", columns="day")
```

`df.melt` — столбцы в строки: длинная форма

```
df.melt(id_vars=["city", "day"]).shape → (4, 4)
```

`.unstack` — внутренний уровень индекса — в столбцы

```
df.groupby(["city", "day"]).size().unstack()
```

`df.set_index` — столбец в индекс; `reset_index()` — обратно

```
df.set_index("city").loc["Сочи", "t"] → 14.0
```

Сортировать

`df.sort_values` — по столбцу; `ascending=False` — по убыванию

```
df.sort_values("t")["city"].iloc[0] → Тула
```

`by=[a, b]` — порядок равных задаёт второй столбец

```
df.sort_values(["city", "t"])
```

`na_position=` — куда пропуски: `"last"` или `"first"`

```
df.sort_values("t", na_position="first")
```

`df.sort_index` — по индексу

```
df.sort_index(ascending=False).index[0] → 3
```

Сохранить

`df.to_csv` — в файл; `index=False` — без индекса

```
df.to_csv("otvet.csv", index=False)
```

`float_format=` — как писать дробные

```
df.to_csv("o.csv", index=False, float_format="%.2f")
```

`s.tolist` — в список; `to_numpy()` — в массив numpy

```
df["t"].to_numpy().dtype → float64
```